epoll_pool

```
/**
 * @brief epoll基于非阻塞I/O事件驱动
 * xwp_fullstack@163.com
 */
#include <stdio.h>
#include <sys/socket.h>
#include <sys/epoll.h>
#include <arpa/inet.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>


#define MAX_EVENTS  1024
#define BUFLEN 128
#define SERV_PORT   8080



/*
 * status:1表示在监听事件中，0表示不在
 * last_active:记录最后一次响应时间,做超时处理
 */
struct myevent_s {
    int fd;                 //cfd listenfd
    int events;             //EPOLLIN  EPLLOUT
    void *arg;              //指向自己结构体指针
    void (*call_back)(int fd, int events, void *arg);
    int status;
    char buf[BUFLEN];
    int len;
    long last_active;       //超时处理
};


int g_efd;          /* epoll_create返回的句柄 */
struct myevent_s g_events[MAX_EVENTS+1];   /* +1 最后一个用于 listen fd */

```

```
41
42 void eventset(struct myevent_s *ev, int fd, void (*call_back)(int, int, void *), voi
43 {
44     ev->fd = fd;
45     ev->call_back = call_back;
46     ev->events = 0;
47     ev->arg = arg;
48     ev->status = 0;
49     //memset(ev->buf, 0, sizeof(ev->buf));
50     //ev->len = 0;
51     ev->last_active = time(NULL);
52
53
54     return;
55 }
56
57
58 void recvdata(int fd, int events, void *arg);
59 void senddata(int fd, int events, void *arg);
60
61
62 void eventadd(int efd, int events, struct myevent_s *ev)
63 {
64     struct epoll_event epv = {0, {0}};
65     int op;
66     epv.data.ptr = ev;
67     epv.events = ev->events = events;
68
69
70     if (ev->status == 1) {
71         op = EPOLL_CTL_MOD;
72     }
73     else {
74         op = EPOLL_CTL_ADD;
75         ev->status = 1;
76     }
77
78
79     if (epoll_ctl(efd, op, ev->fd, &epv) < 0)
80         printf("event add failed [fd=%d], events[%d]\n", ev->fd, events);
81     else
82         printf("event add OK [fd=%d], op=%d, events[%0X]\n", ev->fd, op, events);
83
```

```
84
85      return;
86  }
87
88
89  void eventdel(int efd, struct myevent_s *ev)
90  {
91      struct epoll_event epv = {0, {0}};
92
93
94      if (ev->status != 1)
95          return;
96
97
98      epv.data.ptr = ev;
99      ev->status = 0;
100     epoll_ctl(efd, EPOLL_CTL_DEL, ev->fd, &epv);
101
102
103     return;
104 }
105
106
107
108
109 void acceptconn(int lfd, int events, void *arg)
110 {
111     struct sockaddr_in cin;
112     socklen_t len = sizeof(cin);
113     int cfd, i;
114
115
116     if ((cfd = accept(lfd, (struct sockaddr *)&cin, &len)) == -1) {
117         if (errno != EAGAIN && errno != EINTR) {
118             /* 暂时不做出错处理 */
119         }
120         printf("%s: accept, %s\n", __func__, strerror(errno));
121         return;
122     }
123
124
125     do {
126         for (i = 0; i < MAX_EVENTS; i++) {
127             if (g_events[i].status == 0)
```

```
                break;
        }


        if (i == MAX_EVENTS) {
            printf("%s: max connect limit[%d]\n", __func__, MAX_EVENTS);
            break;
        }


        int flag = 0;
        if ((flag = fcntl(cfd, F_SETFL, O_NONBLOCK)) < 0)
        {
            printf("%s: fcntl nonblocking failed, %s\n", __func__, strerror(errno));
            break;
        }


        eventset(&g_events[i], cfd, recvdata, &g_events[i]);
        eventadd(g_efd, EPOLLIN, &g_events[i]);
    } while(0);


    printf("new connect [%s:%d][time:%ld], pos[%d]\n", inet_ntoa(cin.sin_addr), ntoh


    return;
}


void recvdata(int fd, int events, void *arg)
{
    struct myevent_s *ev = (struct myevent_s *)arg;
    int len;


    len = recv(fd, ev->buf, sizeof(ev->buf), 0);
    eventdel(g_efd, ev);


    if (len > 0) {
        ev->len = len;
        ev->buf[len] = '\0';
```

```
171         printf("C[%d]:%s\n", fd, ev->buf);
172         /* 转换为发送事件 */
173         eventset(ev, fd, senddata, ev);
174         eventadd(g_efd, EPOLLOUT, ev);
175     }
176     else if (len == 0) {
177         close(ev->fd);
178         /* ev-g_events 地址相减得到偏移元素位置 */
179         printf("[fd=%d] pos[%d], closed\n", fd, ev - g_events);
180     }
181     else {
182         close(ev->fd);
183         printf("recv[fd=%d] error[%d]:%s\n", fd, errno, strerror(errno));
184     }
185 
186 
187     return;
188 }
189 
190 
191 void senddata(int fd, int events, void *arg)
192 {
193     struct myevent_s *ev = (struct myevent_s *)arg;
194     int len;
195 
196 
197     len = send(fd, ev->buf, ev->len, 0);
198     //printf("fd=%d\tev->buf=%s\ttev->len=%d\n", fd, ev->buf, ev->len);
199     //printf("send len = %d\n", len);
200 
201 
202     eventdel(g_efd, ev);
203     if (len > 0) {
204         printf("send[fd=%d], [%d]%s\n", fd, len, ev->buf);
205         eventset(ev, fd, recvdata, ev);
206         eventadd(g_efd, EPOLLIN, ev);
207     }
208     else {
209         close(ev->fd);
210         printf("send[fd=%d] error %s\n", fd, strerror(errno));
211     }
212 
213 
214     return;
```

```
}


void initlistensocket(int efd, short port)
{
    int lfd = socket(AF_INET, SOCK_STREAM, 0);
    fcntl(lfd, F_SETFL, O_NONBLOCK);
    eventset(&g_events[MAX_EVENTS], lfd, acceptconn, &g_events[MAX_EVENTS]);
    eventadd(efd, EPOLLIN, &g_events[MAX_EVENTS]);


    struct sockaddr_in sin;


    memset(&sin, 0, sizeof(sin));
    sin.sin_family = AF_INET;
    sin.sin_addr.s_addr = INADDR_ANY;
    sin.sin_port = htons(port);


    bind(lfd, (struct sockaddr *)&sin, sizeof(sin));


    listen(lfd, 20);


    return;
}


int main(int argc, char *argv[])
{
    unsigned short port = SERV_PORT;


    if (argc == 2)
        port = atoi(argv[1]);


    g_efd = epoll_create(MAX_EVENTS+1);


    if (g_efd <= 0)
        printf("create efd in %s err %s\n"    func    strerror(errno));
```

```
        printf("create efd in %s err %s\n", __func__, strerror(errno));
259
260
261     initlistensocket(g_efd, port);
262
263
264     /* 事件循环 */
265     struct epoll_event events[MAX_EVENTS+1];
266
267
268     printf("server running:port[%d]\n", port);
269     int checkpos = 0, i;
270     while (1) {
271         /* 超时验证，每次测试100个链接，不测试listenfd 当客户端60秒内没有和服务器通信，则关
272         long now = time(NULL);
273         for (i = 0; i < 100; i++, checkpos++) {
274             if (checkpos == MAX_EVENTS)
275                 checkpos = 0;
276             if (g_events[checkpos].status != 1)
277                 continue;
278             long duration = now - g_events[checkpos].last_active;
279             if (duration >= 60) {
280                 close(g_events[checkpos].fd);
281                 printf("[fd=%d] timeout\n", g_events[checkpos].fd);
282                 eventdel(g_efd, &g_events[checkpos]);
283             }
284         }
285         /* 等待事件发生 */
286         int nfd = epoll_wait(g_efd, events, MAX_EVENTS+1, 1000);
287         if (nfd < 0) {
288             printf("epoll_wait error, exit\n");
289             break;
290         }
291         for (i = 0; i < nfd; i++) {
292             struct myevent_s *ev = (struct myevent_s *)events[i].data.ptr;
293             if ((events[i].events & EPOLLIN) && (ev->events & EPOLLIN)) {
294                 ev->call_back(ev->fd, events[i].events, ev->arg);
295             }
296             if ((events[i].events & EPOLLOUT) && (ev->events & EPOLLOUT)) {
297                 ev->call_back(ev->fd, events[i].events, ev->arg);
298             }
299         }
300     }
301
```

```
302
303     /* 退出前释放所有资源 */
304     return 0;
305 }
```